

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

CO3061 - INTRODUCTION TO ARTIFICIAL INTELLIGENCE
HOMEWORK 3 PROJECT REPORT

TRAVELING SALESMAN PROBLEM
USING GENETIC ALGORITHM AND SIMULATED ANNEALING

Lecturer: Ph.D. Nguyen Quoc Minh

Class: L0X - HK252

Topic: Topic 1 of Homework 3

Group: [Update your group number]

Members:

1. [Full name] - Student ID: [ID]
2. [Full name] - Student ID: [ID]
3. [Full name] - Student ID: [ID]

Ho Chi Minh City, May 2026

Contents

- 1 Introduction
- 2 Motivations
- 3 Related Work
- 4 Background and Theoretical Foundations
- 5 Methodologies
- 6 Results and Discussion
- 7 Challenges and Limitations
- 8 Conclusion and Future Works
- References

1 Introduction

The Traveling Salesman Problem (TSP) requires finding a minimum-length cycle that visits all cities exactly once and returns to the starting city. In this homework, the dataset contains 51 cities and two forbidden edges: (1,22) and (1,32). The project therefore has to solve not only a permutation optimization problem but also a route feasibility problem.

The objectives are to implement a Genetic Algorithm (GA), plot the final route, solve the same dataset with Simulated Annealing (SA), and compare the two methods in terms of solution quality and runtime.

2 Motivations

GA is an appropriate main method because the search space is factorial in size and the assignment explicitly asks for the design of fitness, selection, crossover, and mutation. The chromosome representation is naturally permutation-based, which makes classical TSP-specific genetic operators directly applicable.

SA is used as a comparison baseline because it is a widely known metaheuristic with a very different search behavior. While GA evolves a population, SA improves a single route and accepts occasional uphill moves to escape local minima.

3 Related Work

- Exact methods such as branch-and-bound and integer programming can prove optimality but are not the focus of this homework.
- Constructive heuristics such as nearest neighbor are fast but usually suboptimal.
- Local search methods such as 2-opt and 3-opt are strong route improvers.
- Metaheuristics such as GA, SA, Ant Colony Optimization, and Tabu Search are common practical choices for medium to large TSP instances.

4 Background and Theoretical Foundations

Each city is represented by Cartesian coordinates. The distance between city i and city j is computed by the Euclidean formula $d(i,j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. A valid route starts at city 1, visits the other 50 cities exactly once, returns to city 1, and avoids the forbidden edges (1,22) and (1,32).

The implemented chromosome stores only the order of the 50 non-start cities. The real route is reconstructed as [1, chromosome..., 1]. This makes the search space more compact and simplifies constraint handling because only the first and last genes can violate the forbidden-edge constraint.

5 Methodologies

5.1 System organization

- solver.py handles dataset parsing, the TSP model, GA, and SA.
- run_experiment.py executes multiple trials, stores results, and plots convergence curves and routes.

- The report builder consumes the generated results instead of re-implementing the solver logic.

5.2 Genetic Algorithm

The initial population combines one nearest-neighbor seed, several mutated variants of that seed, and random feasible permutations. This produces a population that is neither too weak nor too homogeneous at the start.

Fitness is defined as $1 / (1 + \text{route_length})$. Infeasible routes receive fitness 0. In practice, the implementation compares route lengths directly during parent selection because TSP is a minimization problem.

- Selection: tournament selection with tournament size 5.
- Crossover: Order Crossover (OX) to preserve permutation validity.
- Mutation: one of swap, insert, or inversion; followed by an additional inversion with 20% probability.
- Repair: if the first or last city creates a forbidden edge with city 1, it is swapped with a better internal city.
- Diversity control: 16 elites are preserved and 12 immigrants are injected into each generation.

Parameter	Value
Population size	320
Generations	700
Crossover rate	0.95
Mutation rate	0.35
Tournament size	5
Elite size	16
Immigrant count	12

5.3 Simulated Annealing

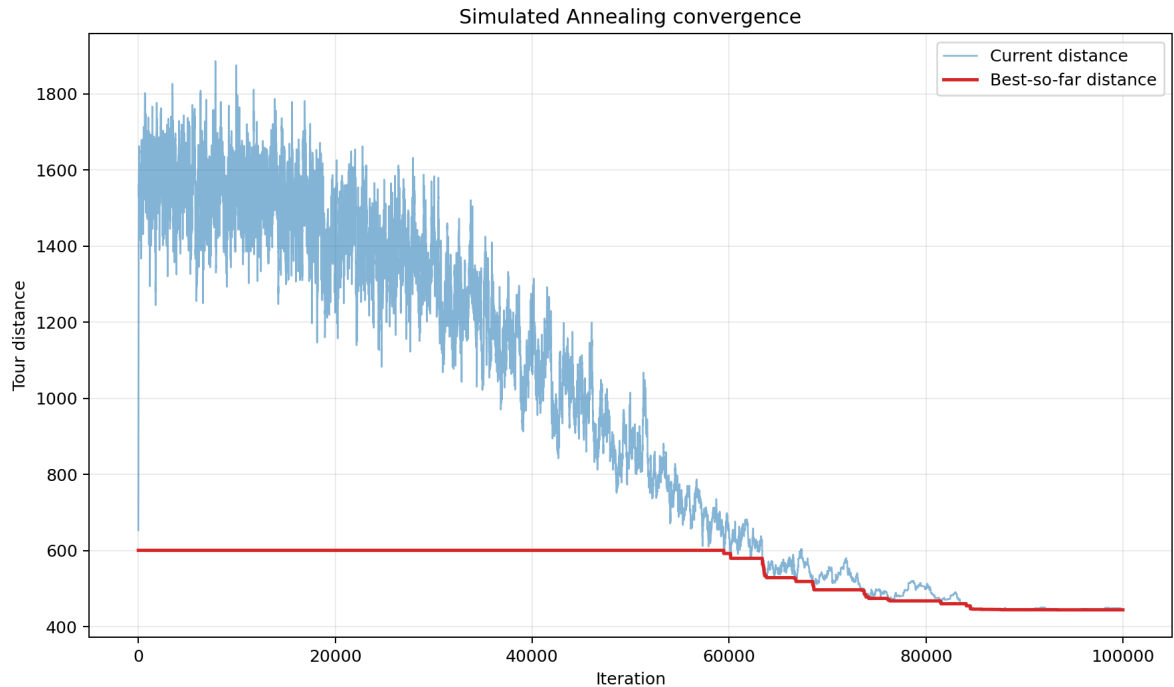
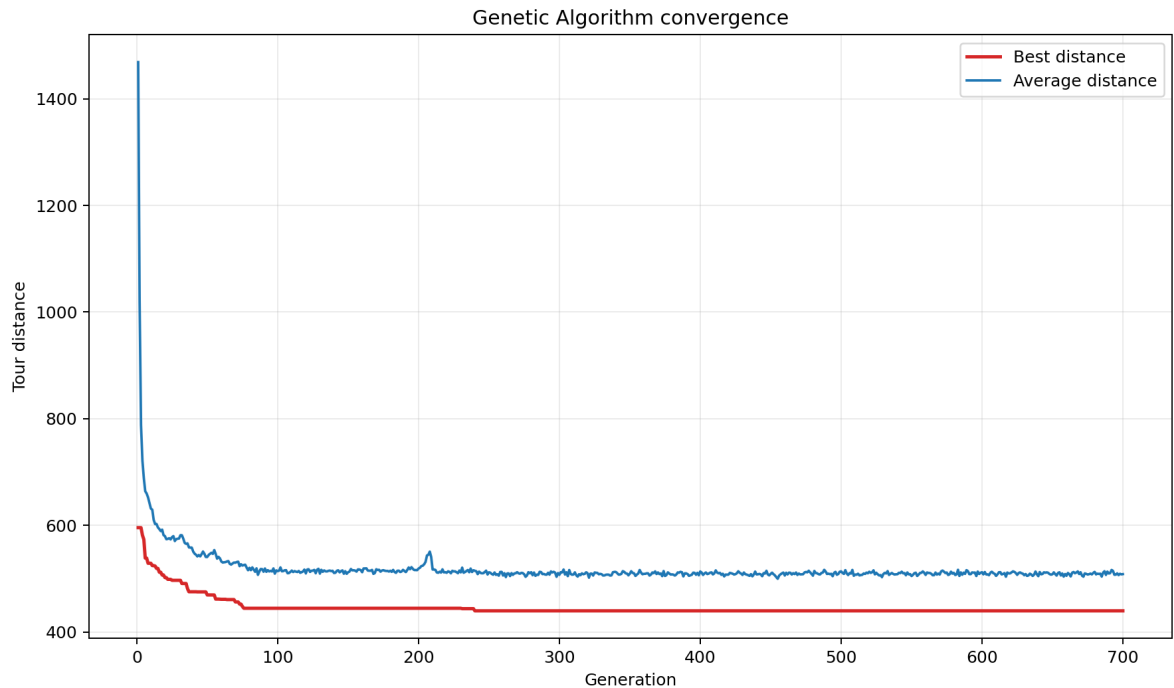
The SA baseline starts from a nearest-neighbor route and uses inversion moves as the neighborhood operator. It accepts a worse move with probability $\exp(-\text{delta} / T)$, where delta is the increase in route cost and T is the temperature. The temperature decreases geometrically until the stopping condition is reached.

6 Results and Discussion

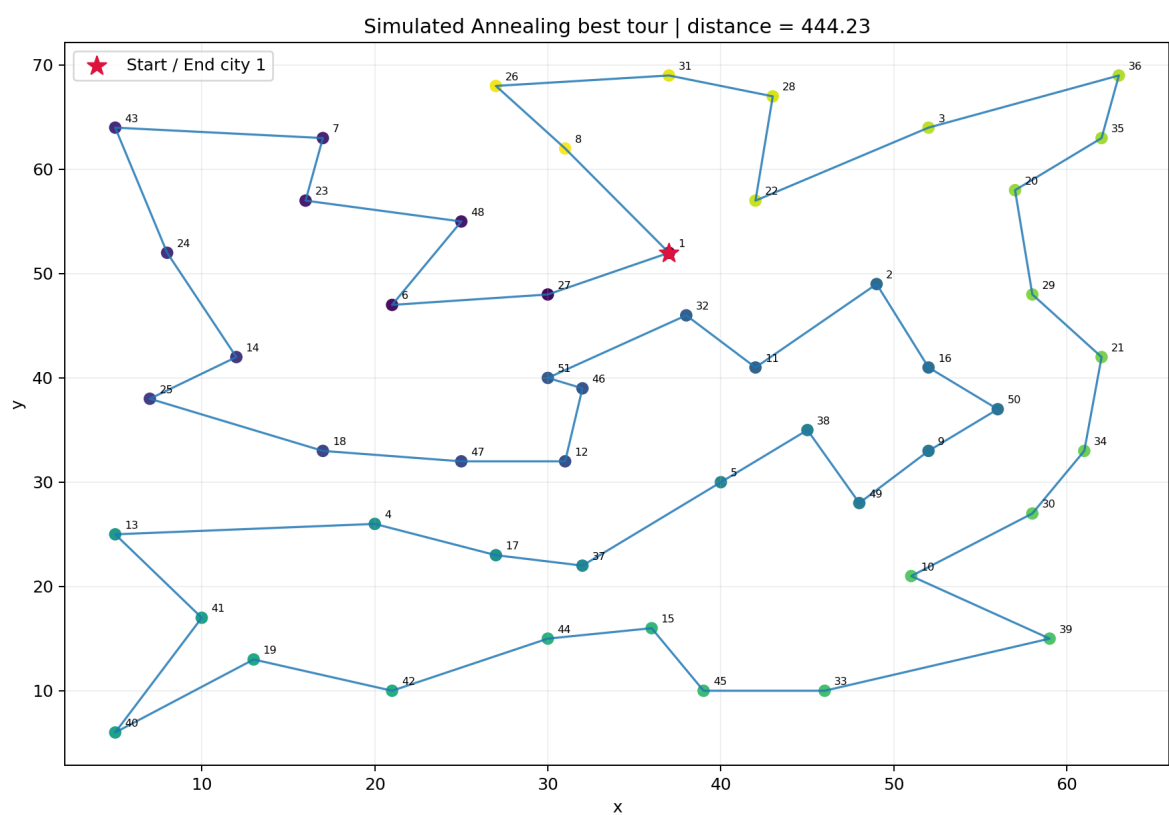
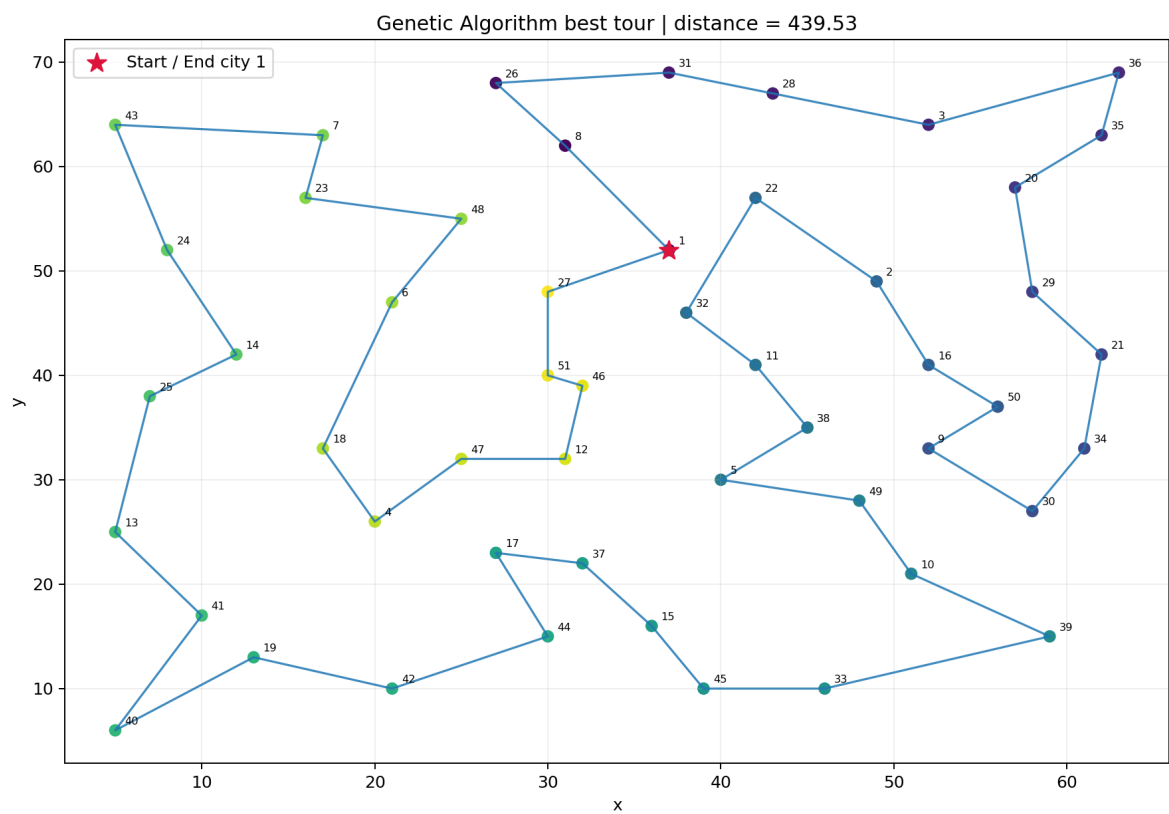
Algorithm	Best distance	Mean distance	Std. distance	Mean runtime (s)
Genetic Algorithm	439.53	440.82	0.81	25.11
Simulated Annealing	444.23	448.36	4.97	6.02

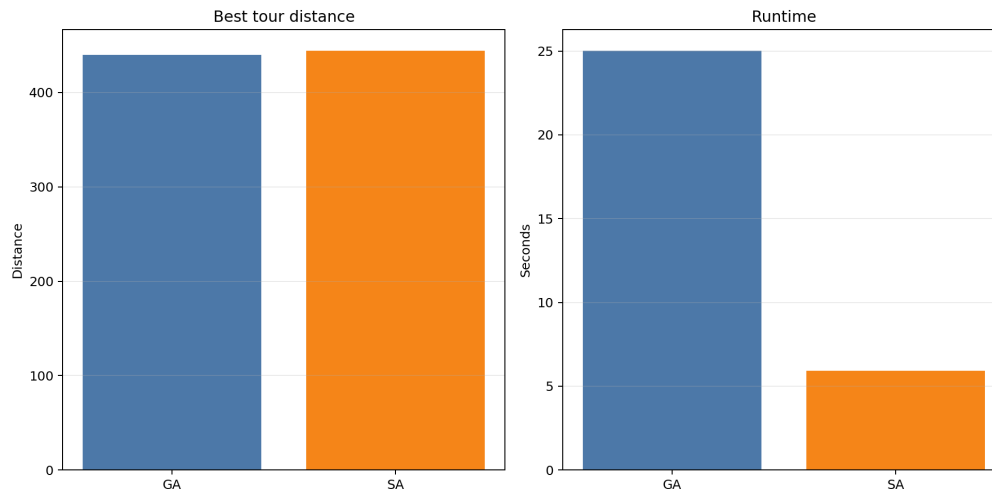
The best GA run found a feasible route of 439.53, while the best SA run found 444.23. GA is slower but more accurate and more stable across repeated trials.

6.1 Convergence curves



6.2 Final routes





The plotted routes confirm that both metaheuristics produce coherent tours. GA shows a slightly better global ordering of cities, which matches its lower final distance.

7 Challenges and Limitations

- The forbidden edges require explicit feasibility handling, especially after crossover and mutation.
- Both GA and SA are stochastic, so repeated runs do not produce identical results.
- The reported solution is a high-quality feasible route, not a proof of global optimality.
- The evaluation uses one dataset only, so the same hyperparameters may not remain best on other TSP instances.

8 Conclusion and Future Works

The project satisfies the Homework 3 requirements by implementing a constrained TSP solver with Genetic Algorithm, adding a Simulated Annealing baseline, plotting final routes, and reporting convergence histories. On the provided dataset, GA achieves the best route length of 439.53, outperforming the best SA distance of 444.23.

- Possible future work includes integrating 2-opt local search into GA.
- Adaptive mutation schedules may further improve convergence.
- Additional baselines such as Ant Colony Optimization could make the comparison broader.
- Testing more datasets would strengthen the experimental claims.

References

- Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Pearson, 2021.
- David E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, 1989.

- S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, Optimization by Simulated Annealing, Science, 1983.
- G. Reinelt, The Traveling Salesman: Computational Solutions for TSP Applications, Springer, 1994.
- Homework 3 handout, CO3061 - Introduction to Artificial Intelligence, Ho Chi Minh City University of Technology, 2026.